

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS700 Research Proposal

Unicode / ASCII Smuggling Attacks in Chatbots

Student number: 26277850

Supervisor(s):
Prof Jan Eloff

Contents

1 Introduction **4**

2 Problem Statement **5**
2.1 The Research Problem . . . 5
2.2 Research Objectives . . . 5
2.3 Scope and Limitations . . . 5

3 Literature Study **6**
3.1 Chatbots and Large Language Models . . . 6
3.2 Adversarial Perturbations: From Pixels to Tokens . . . 6
3.3 Encoding-Based Attacks on Language Models . . . 6
3.4 Systematic Evaluation on Open-Source Chatbots . . . 6
3.5 Targeted Encoding and Token Importance . . . 7
3.6 Transferability of Open-Source Attacks . . . 7
3.7 Why Multi-Turn Context Changes the Attack Surface . . . 7
3.8 Refusal Localization and Why Encoding Attacks Are Effective . . . 7
3.9 Transferability Across Architectures and Tokenizers . . . 8
3.10 Research Gap . . . 8

4 Methodology **9**
4.1 Benchmark Dataset . . . 9
4.2 Target Models . . . 9
4.3 Artefact 1: Unicode/ASCII Mutation Engine . . . 9
4.3.1 Trigger Word Detection . . . 10
4.3.2 Mutation Modes . . . 10
4.4 Artefact 2: End-to-End Exploitation Pipeline . . . 10
4.4.1 Component Mapping . . . 10
4.4.2 Evaluation Method . . . 11
4.5 Experimental Design . . . 11
4.5.1 Multi-Turn Strategy A: Context Priming . . . 11
4.5.2 Multi-Turn Strategy B: Progressive Accumulation . . . 12
4.6 Trial Counts and Compute . . . 12
4.7 Evaluation Metrics . . . 12
4.8 Statistical Analysis . . . 12
4.9 Reproducibility . . . 13

5 Planning **14**

Abstract

Chatbots built on large language models (LLMs) are deployed at scale in consumer and enterprise settings, where they process sensitive user input and execute actions through tool integrations. Modern chatbot safety systems operate across multiple layers: token-level classifiers that evaluate input representations, semantic classifiers that assess meaning, and content filters that scan output. Encoding based attacks are particularly dangerous because they exploit the tokenizer's representation layer, which underpins all of these downstream safety checks, including semantic classifiers, meaning that a perturbation introduced at the tokenization stage propagates through the full safety stack.

Unicode and ASCII encoding manipulations can alter token representations without changing what the text looks like to a human reader, creating a class of imperceptible attacks that remain invisible to oversight while bypassing multiple safety mechanisms simultaneously. Prior work has established that such attacks succeed in single-turn settings against either closed-source or open-source models, but not both, and none of the existing studies evaluates the effect of multi-turn conversation context on attack success rates in open-source chatbot deployments.

This research investigates the susceptibility of open-source chatbots to Unicode and ASCII smuggling attacks across single-turn and multi-turn conversation contexts. Two artefacts will be developed: a Unicode/ASCII Mutation Engine that generates adversarial inputs across four encoding families guided by safety classifier-based trigger word detection, and an end-to-end exploitation pipeline integrated with the PyRIT red-teaming framework. The expected contribution is a structured, empirical characterisation of how encoding-based attacks interact with conversational memory in open-source chatbots, specifically whether multi-turn context and combined encoding families increase attack effectiveness beyond single-turn baselines, together with reusable tooling for future red-teaming research.

Keywords: chatbots, large language models, Unicode smuggling, ASCII smuggling, adversarial attacks, jailbreaking, prompt injection, multi-turn, red-teaming, imperceptible attacks.

1 Introduction

Chatbots powered by large language models have become the primary interface through which organisations and individuals interact with AI systems. Singh and Namin [Sin24] provide a comprehensive overview of how LLM integration has transformed chatbot capabilities: where earlier systems relied on rule-based dialogue management, modern chatbots generate responses from learned representations of language, enabling open-domain conversation at a quality previously unattainable. This shift in capability has been accompanied by a shift in deployment scope. Chatbots now handle customer queries, assist with medical triage, support legal research, and, in agentic configurations, execute code and call external APIs on behalf of users.

The architectural foundation of this capability is the Transformer, introduced by Vaswani et al. [Vas17]. The self-attention mechanism allows the model to relate any token in an input sequence to any other, enabling LLMs to process long-range dependencies in language. Brown et al. [Bro20] demonstrated that a sufficiently large pre-trained Transformer can be adapted to new tasks by feeding examples into its context window, without weight updates. This is the mechanism that makes prompt engineering effective: chatbot behaviour is shaped by what is in the context window at inference time, not only by what was learned during training. The practical consequence is that an adversary who can control what the model processes at the token level can influence what the model does, even if what they submit appears innocuous to a human reviewer.

The adversarial vulnerability of deep neural networks to imperceptible input perturbations was established by Szegedy et al. [Sze14], who showed that networks misclassify inputs when subjected to structured, imperceptible perturbations. Goodfellow, Shlens and Szegedy [Goo15] explained the mechanism: neural networks are locally linear in high dimensions, so perturbations individually below the perception threshold accumulate into a significant shift in the model output. The perturbation per dimension is invisible; the total effect is not. Ebrahimi et al. [Ebr18] showed this logic applies to text: gradient-based token substitutions alter text classification outcomes in white-box settings. The adversarial threat had moved from pixels to tokens.

The transition to the chatbot context followed. Boucher et al. [Bou22] demonstrated that encoding-level manipulations, including Unicode control characters, homoglyphs, and invisible characters, degrade NLP model behaviour in a black-box setting without gradient access. Wunderwuzzi [Wun24] demonstrated ASCII smuggling in practice: Tag Block Unicode characters invisible to human readers carry hidden instructions that chatbots process and act upon. Gao et al. [Gao25] formalised this into imperceptible jailbreaking: variation selector characters produce prompts that look identical to the original malicious input while their tokenization bypasses safety filters. Sarabamoun [Sar25] systematically evaluated these attacks across seven open-source models, covering Unicode, homoglyph, structural, and textual encoding families, confirming broad susceptibility in single-turn settings. Graves [Gra26] extended this to tool-use, demonstrating that chatbots with tool access act on hidden Unicode instructions that tool-less models resist.

Taken together, this body of work establishes that encoding-based attacks on chatbots are real, measurable, and reproducible, but the threat space has not been fully characterised. The specific combination of open-source chatbot targets, multi-turn conversation context, combined encoding families, and trigger word-guided mutation has not been studied together. This research closes that gap. The central argument is that these dimensions interact: multi-turn context allows injected instructions to persist across per-turn safety checks, and the combination of encoding families produces attack variants that individual families cannot. Studying each dimension in isolation, as prior work has done, is insufficient to characterise the threat that users and deployers of open-source chatbots actually face.

2 Problem Statement

2.1 The Research Problem

Open-source chatbots are susceptible to imperceptible Unicode and ASCII smuggling attacks. How this susceptibility changes across multi-turn conversations, and whether combining encoding families increases attack effectiveness, has not been characterised. Without this characterisation, developers cannot assess the actual risk exposure of open-source chatbot deployments, and defenders cannot prioritise mitigations.

Three properties of open-source chatbots make this gap particularly consequential. First, their weights are publicly accessible, enabling white-box adversarial crafting that Zou et al. [Zou23] demonstrated transfers to closed commercial targets. An adversary can develop an attack on an open-source chatbot and deploy it against a commercial one. Second, chatbot safety filters evaluate input per turn; in a multi-turn conversation, injected instructions embedded in an earlier turn may persist in the context window and influence later responses after the filter has already cleared the input. Third, encoding attacks are particularly dangerous because they exploit the tokenizer's representation layer, which underpins all downstream safety mechanisms including both token-level classifiers and semantic safety classifiers, meaning that a single perturbation at the tokenization stage propagates through the full safety stack simultaneously.

2.2 Research Objectives

The main objective of this research is to evaluate and characterise the susceptibility of open-source chatbots to Unicode and ASCII smuggling attacks across single-turn and multi-turn conversation contexts.

The sub-objectives are:

- To implement a Unicode/ASCII Mutation Engine (Artefact 1) that generates adversarial chatbot inputs across four encoding families: Unicode variation selectors, homoglyph substitution, structural Unicode injection, and ASCII smuggling via Tag Block characters, with mutation guided by safety classifier-based trigger word detection.
- To implement an end-to-end exploitation pipeline (Artefact 2), integrated with the PyRIT red-teaming framework, that supports single-turn and multi-turn attack orchestration against open-source chatbot targets.
- To evaluate the attack success rate of each encoding family independently against the target chatbots in single-turn settings, establishing a baseline consistent with the Sarabamoun [Sar25] setting.
- To evaluate whether combining encoding families produces a higher attack success rate than any individual family alone.
- To evaluate whether multi-turn conversation context changes the attack success rate relative to the single-turn baseline, using two distinct multi-turn strategies: context priming and progressive accumulation.
- To analyse whether model scale (7–9B vs. 70–72B parameters) affects susceptibility to encoding-based attacks across architecturally distinct model families.

2.3 Scope and Limitations

This study targets five open-source instruction-tuned chatbots from three model families (Llama, Qwen, Gemma) at two parameter scales, accessed through Ollama and Hugging Face. The attack surface is unimodal text input only; multimodal inputs are excluded. Closed-source commercial chatbots are not primary targets; comparisons to commercial models draw on published results from prior work.

Tool-access configuration is excluded from the current scope. The relationship between encoding attacks and chatbot tool-use, demonstrated qualitatively by Graves [Gra26] on closed-source models, is identified as future work. All attacks are conducted in a controlled local environment with no live production systems or real users involved. The study does not include the development of defences or mitigations; the contribution is the characterisation of the attack surface, which is a prerequisite for defence design.

3 Literature Study

3.1 Chatbots and Large Language Models

Singh and Namin [Sin24] survey the integration of LLMs into chatbot systems, documenting the shift from retrieval-based and rule-based dialogue management to generative, Transformer-based architectures. Their work establishes the current state of chatbot capability and identifies testing and evaluation as a critical open challenge. The Transformer architecture that underlies this capability was introduced by Vaswani et al. [Vas17]: the self-attention mechanism allows any token in a sequence to attend to any other, enabling the model to capture long-range dependencies that recurrent architectures handled poorly. Brown et al. [Bro20] demonstrated that a large pre-trained Transformer generalises to new tasks through in-context learning, where examples provided in the context window shape behaviour without weight updates. This is the mechanism that makes chatbot prompts a meaningful attack surface: controlling what is in the context window at inference time is equivalent to influencing the model's behaviour.

3.2 Adversarial Perturbations: From Pixels to Tokens

The vulnerability of deep neural networks to structured, imperceptible input perturbations was established by Szegedy et al. [Sze14] in the image domain. Goodfellow, Shlens and Szegedy [Goo15] attributed this to the linear nature of neural networks in high-dimensional spaces: the Fast Gradient Sign Method (FGSM) exploits this linearity to construct perturbations that accumulate across input dimensions into a significant output shift, while remaining individually below the perception threshold. Ebrahimi et al. [Ebr18] translated this to text in a white-box setting through their HotFlip method, showing that gradient-based token substitutions alter text classification outcomes using the same underlying logic. This confirmed that adversarial vulnerability is not specific to continuous input spaces and applies to the discrete token sequences that chatbots process.

3.3 Encoding-Based Attacks on Language Models

Boucher et al. [Bou22] extended the adversarial threat to encoding-level manipulations that require no gradient access. By injecting Unicode control characters, invisible characters, and homoglyphs into text inputs, they demonstrated that NLP model behaviour degrades in a black-box setting, which is the position of any real-world chatbot attacker. Their key finding was that a single imperceptible encoding injection is sufficient to significantly reduce model performance, and three injections can functionally break most classifiers.

Wunderwuzzi [Wun24] demonstrated the practical instantiation of this threat in chatbots specifically: Tag Block Unicode characters (U+E0000 through U+E007F) are rendered as invisible by human-readable interfaces but are processed as tokens by language models, allowing hidden instructions to be embedded in text that appears clean to any human reviewer. This demonstration grounded the theoretical attack class in observable chatbot behaviour.

Gao et al. [Gao25] formalised imperceptible jailbreaking against LLMs, demonstrating that Unicode variation selectors appended to prompt tokens produce inputs that are visually identical to the original malicious question while their tokenization is altered in a way that bypasses safety filters. Their experiments achieved high attack success rates against four aligned LLMs in single-turn settings on closed-source models.

3.4 Systematic Evaluation on Open-Source Chatbots

Sarabamoun [Sar25] conducted the most comprehensive empirical study of special-character adversarial attacks against open-source language models to date, evaluating Unicode, homoglyph, structural, and textual encoding attacks across seven prominent open-source models. The study confirmed broad susceptibility across model families and provided empirical attack success rates for each encoding family. All evaluations were conducted in single-turn settings, leaving the multi-turn dimension and the effect of combining encoding families uncharacterised for open-source deployments.

3.5 Targeted Encoding and Token Importance

TriageFuzz (arXiv:2603.23269, 2026) validated the principle that not all tokens contribute equally to refusal behaviour, and that targeting high-importance tokens achieves comparable attack success with significantly fewer queries. Their approach identifies importance via attention head ablation in white-box settings. This study adopts the same principle but uses Llama Guard 3 as a dedicated safety classifier surrogate to identify trigger words, which is more generalisable across model families and requires no white-box access to the target model.

3.6 Transferability of Open-Source Attacks

Zou et al. [Zou23] demonstrated that adversarial inputs crafted against open-source models transfer to closed commercial models. The public availability of open-source chatbot weights enables an adversary to develop attacks with full white-box access and deploy them against commercial targets. This also supports the transferability of trigger word identification in this study: safety-critical tokens identified by one model's safety mechanism are expected to generalise across architecturally distinct models, because harmful semantics reflect shared patterns from safety alignment training across the field.

3.7 Why Multi-Turn Context Changes the Attack Surface

The intuition that multi-turn conversations are more vulnerable than single-turn interactions is confirmed mechanistically by recent work. Li et al. [Li26] study safety failures from a state-space perspective and show that many multi-turn failures arise from structured contextual state evolution rather than isolated prompt vulnerabilities. Models that appear robust under static evaluation undergo rapid and reproducible safety collapse under structured multi-turn interaction, with mechanistic analysis revealing monotonic drift away from refusal-related representations as dialogue history accumulates. This is the mechanism this study exploits in Strategy A: the context window accumulates state that progressively degrades the safety boundary.

Owiredu-Ashley [Owi26] confirms this empirically through continuous per-round compliance measurement, demonstrating that safety properties degrade measurably under sustained adversarial interaction and that per-turn evaluation is insufficient to characterise safety across a full conversation. Taken together, these two papers establish that safety alignment behaves differently over conversational trajectories than under isolated query evaluation, which is precisely the gap this study targets.

At the empirical level, Reddy, Zagula and Saban [Red25] show that multi-turn attacks achieve 95% ASR on Llama-3.1-8B within six turns, a 24% improvement over single-turn baselines on the same model. This is the strongest published empirical evidence that multi-turn context amplifies attack effectiveness and directly motivates the multi-turn dimension of this study.

The psychological mechanism underlying Strategy A (context priming) is formalised by Kumarappan and Mujoo [Kum25b], who operationalise the Foot-in-the-Door principle into reproducible multi-turn attack templates: a small, benign initial request establishes a compliant conversational context that makes the model more likely to comply with a subsequent larger, harmful request. Their evaluation across seven models from three LLM families confirms that contextual history systematically increases attack success beyond what encoding alone explains.

3.8 Refusal Localization and Why Encoding Attacks Are Effective

Understanding why encoding-based attacks work requires understanding where refusal behaviour lives in a language model. Arditi et al. (2024) demonstrated that refusal is encoded in a single latent direction in the model's residual stream, and that suppressing this direction removes refusal entirely while preserving all other capabilities. This is the mechanism behind abliterated or "heretic" model variants: they function normally for all tasks except refusing, because the refusal mechanism was concentrated in a removable component rather than distributed across the network. An intervention that destroys distributed capabilities would degrade the model broadly; the fact that it does not provides strong empirical evidence for refusal localization.

Yin et al. [Yin26] provide complementary evidence from the opposite direction: layer-selective optimal transport intervention at one or two carefully chosen layers at approximately 40–60% network depth sub-

stantially outperforms full-network interventions for refusal ablation, confirming that refusal mechanisms are localized rather than distributed. Ji et al. [Ji25] further show that refusal mechanisms are governed by functionally independent directions in activation space, confirming that multiple distinct but localized pathways drive refusal behaviour.

This architecture has a direct implication for encoding attacks. Abliteration removes refusal by modifying the weights that implement those localized layers; encoding-based attacks achieve the same outcome from the input side. By corrupting the token representations that feed into refusal-sensitive layers, encoding perturbations prevent the safety mechanism from activating without requiring any access to model weights. The two approaches target the same component from opposite directions. The inclusion of random-mutation baselines in this study allows empirical evaluation of whether targeted mutation provides advantage over uniform perturbation: if refusal is strongly distributed, targeted mutation should show limited advantage, and this will be directly observable in the ASR comparison.

3.9 Transferability Across Architectures and Tokenizers

The use of Llama Guard 3 as a single surrogate classifier to identify trigger words across all five target models rests on the transferability assumption: safety-critical tokens identified by one model's safety mechanism generalise to other architecturally distinct models. This assumption is supported at multiple levels.

Kumar et al. [Kum25] show that proxy-based adversarial approaches approach white-box effectiveness even with minimal target knowledge, meaning that trigger importance estimates from a surrogate carry signal for targets the surrogate has never seen. Zheng et al. [Zhe25] specifically address transfer across heterogeneous architectures, confirming that adversarial examples transfer from surrogate models to unseen black-box targets despite architectural differences. The foundational survey by Zhao et al. [Zha23] establishes the theoretical and empirical basis for this cross-architecture transferability.

Different model families (Llama, Qwen, Gemma) use different tokenization schemes and may apply different Unicode normalisation behaviours, which could affect how encoding perturbations fragment tokens internally. The three-family model selection in this study makes this testable: if tokenizer diversity substantially weakens transferability, this will appear as reduced ASR on models architecturally distant from Llama Guard 3. The study does not assume perfect transferability; it measures it, and the architecture comparison becomes one of the study's empirical contributions.

3.10 Research Gap

The literature has studied the dimensions of encoding-based chatbot attacks independently. No existing study evaluates the combination of open-source chatbot targets, multiple encoding families used together, trigger word-guided mutation, and multi-turn conversation context. Table 1 positions this study relative to related work.

Table 1: Positioning of this study relative to related work.

Study	Target	Encoding Families	Multi-Turn	Targeted
Gao et al. [Gao25]	Closed-source	Variation selectors	No	No
Sarabamoun [Sar25]	Open-source (7)	Unicode, homoglyph, structural, textual	No	No
Graves [Gra26]	Closed-source	2 families	No	No
This study	**Open-source (5)**	**All 4 + combinations**	**Yes**	**Yes**

4 Methodology

This research uses a prototype and experiment methodology. Two software artefacts are developed and used to conduct a structured set of experiments that produce empirical evidence to answer the research questions. This is the standard approach for applied security research: when the research questions require empirical evidence from running systems, constructing purpose-built tooling and evaluating it under controlled conditions is the appropriate method.

4.1 Benchmark Dataset

All experiments use the JailbreakBench Behaviors dataset (JBB-Behaviors) [Cha24], a standard red-teaming benchmark comprising 100 distinct misuse behaviours across 10 harm categories aligned with OpenAI's usage policies. The dataset is composed of 55% original examples with the remainder sourced from AdvBench and HarmBench. JBB-Behaviors is the most widely cited jailbreak benchmark in 2025–2026 adversarial LLM papers, and its use ensures results are directly comparable to the broader literature.

4.2 Target Models

Five open-source instruction-tuned chatbots are evaluated, drawn from three architecturally distinct families at two parameter scales. The selection isolates the effect of model scale from architecture: Llama and Qwen are each represented at both scales, while Gemma provides a third architecture at the small scale.

Table 2: Target chatbot models.

Model	Parameters	Deployment	Family
Llama-3.1-8B-Instruct	8B	Local (Ollama)	Llama
Qwen2.5-7B-Instruct	7B	Local (Ollama)	Qwen
Gemma-2-9B-it	9B	Local (Ollama)	Gemma
Llama-3.1-70B-Instruct	70B	Ollama Cloud	Llama
Qwen2.5-72B-Instruct	72B	Ollama Cloud	Qwen

4.3 Artefact 1: Unicode/ASCII Mutation Engine

The Mutation Engine is a Python software tool implemented as a `PromptConverter` within the PyRIT framework (`SemanticUnicodeConverter`). It takes a seed prompt as input and produces a set of adversarially mutated variants by applying encoding manipulations guided by trigger word detection. The engine implements four encoding families:

- Homoglyph substitution [Bou22, Sar25]: Replacing ASCII characters in trigger words with visually identical Unicode homoglyphs from Cyrillic, Greek, and other scripts. The tokenizer encounters a mixed-script sequence rare in training corpora, forcing unfamiliar subword splits. The safety classifier never sees the token pattern it was trained to flag.
- Unicode variation selectors [Gao25]: Appending invisible variation selector characters (U+FE00–U+FE0F; U+E0100–U+E01EF) after each character in the trigger word. These characters are zero-width and do not render visually, but tokenizers encode them as separate tokens, completely fragmenting the token pattern the safety classifier was trained to recognise.
- Structural Unicode injection [Bou22, Sar25]: Inserting invisible zero-width characters, specifically zero-width joiners (U+200D), zero-width non-joiners (U+200C), and bidirectional override characters, between the letters of trigger words. This fragments words at the token level through insertion rather than replacement, operating on a distinct mechanism from homoglyphs and testing whether defences against substitution protect against fragmentation.
- ASCII smuggling via Tag Block characters [Wun24, Gra26]: Encoding the entire harmful instruction in Tag Block Unicode characters (U+E0000–U+E007F) and appending it to a benign-looking visible prompt. Unlike the other three families, which disguise trigger words in-place, this family hides the

full harmful payload in an invisible channel. The safety filter evaluates only the benign visible text, while the model tokenizes and processes the hidden instruction.

4.3.1 Trigger Word Detection

Before any experiment runs, each of the 100 seed prompts is processed through a trigger word detection pipeline using Llama Guard 3 (8B, run locally via Ollama) as a safety classifier surrogate. The detection pipeline identifies the minimal set of words whose removal causes the classifier to reclassify a prompt from unsafe to safe. These words represent the tokens that carry harmful semantic intent and are the targets for encoding mutation.

The procedure follows a hierarchical removal approach:

1. Single-word removal: Each word is removed in turn and the prompt is re-classified. If removing a single word flips the classification to safe, that word is the trigger.
2. Pairwise removal: If no single word is sufficient, pairs of content words are tested. The first pair whose removal flips the classification identifies the interacting trigger words.
3. Triple removal: If pairs are insufficient, triples of content words are tested.
4. Fallback: If no combination up to triples flips the classification, all content words are treated as encoding targets.

This targeted approach concentrates encoding disruption on the specific tokens that trigger safety mechanisms, maximising imperceptibility while focusing the attack where it matters. Unlike the uniform or random encoding applied by Sarabamoun [Sar25], which modifies the entire prompt, this approach leaves the majority of the prompt as clean, readable text.

The transferability of trigger word identification across target models is supported by Zou et al. [Zou23]: safety-critical tokens identified by Llama Guard 3 are expected to generalise across the Llama, Qwen, and Gemma families, because harmful semantics reflect shared patterns from safety alignment training across the field. Processing all 100 prompts takes approximately 15–30 minutes and is a one-time cost before experiments begin.

4.3.2 Mutation Modes

For each encoding family, the engine supports:

- Targeted mutation: Applied only to the detected trigger words.
- Random mutation: Applied at randomly selected positions across the prompt, providing a comparison baseline for the targeted approach.

Five variants per encoding family per prompt are generated (targeted and random combinations), producing 500 variants per model per experiment condition.

4.4 Artefact 2: End-to-End Exploitation Pipeline

The exploitation pipeline integrates Artefact 1 with PyRIT (Python Risk Identification Toolkit), Microsoft's open-source red-teaming framework, which natively supports single-turn and multi-turn attack orchestration. The pipeline maps all components to PyRIT's existing abstractions, requiring no modifications to the framework itself.

4.4.1 Component Mapping

Three distinct LLM roles operate in the pipeline: the target chatbot is the safety-aligned open-source model being attacked; the scorer (judge) is a strong general model that evaluates whether a target response violates safety policy; and the adversarial chat generates attack prompts in multi-turn mode using an uncensored model. Llama Guard 3 operates separately as a classifier during trigger word detection only.

All model connections use `OpenAIChatTarget`, which connects to any OpenAI-compatible API endpoint. Despite the class name, no commercial OpenAI service is used in this study: all endpoints point to

Table 3: Pipeline components and PyRIT abstractions.

Component	PyRIT Abstraction	Role
Trigger Word Detector	`auxiliary_attacks`	Pre-processing
Unicode Encoder (Artefact 1)	`PromptConverter`	Mutation
Target Chatbot	`OpenAIChatTarget`	Attack target
Scorer (Judge)	`SelfAskTrueFalseScorer`	Evaluation
Adversarial Chat	`OpenAIChatTarget` in `AttackAdversarialConfig`	Multi-turn prompt generation
Single-turn	`PromptSendingAttack`	Experiments 1 & 2
Multi-turn	`RedTeamingAttack`	Experiment 3a & 3b

Ollama (local) or Ollama Cloud, both configured via the `~/.pyrit/.env` file. Swapping to any other OpenAI-compatible provider requires only an endpoint and key change in that file.

4.4.2 Evaluation Method

Attack success is evaluated using PyRIT's built-in `SelfAskTrueFalseScorer`, which uses a judge prompt to classify whether a target model's response provides harmful content. A rule-based keyword evaluator runs in parallel as a secondary check. Disagreements between the two scorers are flagged for manual inspection. A random sample of 50–100 responses across all experiments will be manually verified by the researcher to report human–scorer agreement and validate the automated evaluation pipeline. This addresses the known reliability concerns with LLM-as-a-judge approaches [Sch26].

4.5 Experimental Design

Three experiments are conducted (Experiment 3 has two sub-strategies), each building on the previous. Table 4 summarises the design.

Table 4: Experimental design overview.

Exp	Condition	What It Tests
1	Single-turn, individual encoding families	Baseline ASR per family, replicates Sarabamoun [Sar25] on this model set
2	Single-turn, combined encoding families	Whether combining families produces higher ASR than any individual family
3a	Multi-turn, Strategy A (context priming)	Whether benign conversational context before a harmful request increases ASR
3b	Multi-turn, Strategy B (progressive accumulation)	Whether progressively increasing encoding density across turns degrades safety alignment

4.5.1 Multi-Turn Strategy A: Context Priming

Hypothesis: Establishing benign conversational context before delivering an encoded harmful request increases ASR relative to single-turn delivery of the same encoded prompt.

The attack unfolds over a fixed 5-turn budget: turns 1–3 establish benign context that narrows the topic toward the target harm domain without using harmful keywords; turn 4 delivers the harmful request with trigger words encoded; turn 5 delivers a rephrased follow-up if turn 4 was refused. The adversarial chat model generates turns using a custom YAML template (`unicode_smuggling_context_priming.yaml`) that instructs it to prime the target without triggering refusal in the early turns.

For each of the 10 harm categories in JBB-Behaviors, 4 hand-crafted benign context templates are prepared (40 templates total). Strategy A results are compared against Experiment 1 results for the same encoded prompt delivered without prior context, isolating the effect of context priming from encoding.

4.5.2 Multi-Turn Strategy B: Progressive Accumulation

Hypothesis: Progressively increasing encoding density across turns degrades safety alignment more than a single-turn attack with the same final encoding density.

Over 5 turns, the same harmful request is delivered with increasing encoding density: turn 1 encodes only the primary trigger word; turn 2 encodes the top two trigger words; turn 3 encodes all trigger words; turn 4 applies full encoding to the entire prompt; turn 5 applies combined encoding families. Strategy B results are compared against Experiment 1 at the equivalent encoding density, isolating the effect of progressive exposure from encoding density alone.

4.6 Trial Counts and Compute

Table 5: Experiment-level interaction counts.

Experiment	Calculation	Interactions
Exp 1: Single-turn, individual families	$100 \times 4 \times 5 \times 5$	10,000
Exp 2: Single-turn, combined families	$100 \times 6 \times 3 \times 5$	9,000
Exp 3a: Multi-turn Strategy A (5 turns)	$100 \times 4 \times 3 \times 5 \times 5$	30,000
Exp 3b: Multi-turn Strategy B (5 turns)	$100 \times 4 \times 3 \times 5 \times 5$	30,000
Total		**≈79,000**

Estimated compute at greedy decoding on an RTX 4060 (local models) and Ollama Cloud (large models) is approximately 150–200 hours for single-turn experiments, equivalent to 6–8 days of continuous overnight runs. Multi-turn experiments (3a and 3b) will take longer per interaction because the adversarial chat model generates prompts using a thinking-enabled uncensored model, which adds reasoning overhead of approximately 2–5 seconds per turn beyond the target model inference time. The total estimate including this overhead is 200–250 hours; this is still feasible within the August–September experimental window scheduled as overnight runs.

4.7 Evaluation Metrics

- Attack Success Rate (ASR): Primary metric across all experiments. The proportion of mutated variants that produce a response classified as a safety policy violation by `SelfAskTrueFalseScorer`. 95% confidence intervals are reported around each ASR value.
- Imperceptibility score: Fraction of mutated characters that are visually indistinguishable from the original. Reported per encoding family.
- Turn depth at first success: In Experiments 3a and 3b, the conversation turn at which a jailbreak is first achieved.
- Encoding family effectiveness ranking: Ranked comparison of ASR across the four encoding families and their combinations, per model and in aggregate.

4.8 Statistical Analysis

McNemar's test [McN47] is used for all paired comparisons between experimental conditions (e.g., single-turn vs. multi-turn, individual vs. combined encoding). McNemar's test is the appropriate choice because the same 100 prompts are evaluated in each condition, making the observations paired binary outcomes (attack succeeded or failed). Significance is evaluated at $p < 0.05$. Effect sizes are reported as odds ratios. Without statistical testing, observed differences in ASR between conditions (e.g., 60% vs. 68%) cannot be attributed to the experimental manipulation rather than random variation; this is a standard requirement in adversarial ML evaluation.

4.9 Reproducibility

All experiments use greedy decoding (temperature = 0.0) to produce deterministic outputs: the same input always produces the same response, eliminating stochastic variation and meaning a single run per variant is sufficient. All model versions are pinned to specific Ollama model tags (e.g., `llama3.1:8b-instruct-q4_K_M`). Local models run at Q4_K_M quantisation on an NVIDIA RTX 4060 (8 GB VRAM). All seed prompts, encoding configurations, and experimental parameters are logged via PyRIT's built-in logging and will be released as supplementary material.

Table 6: Fixed inference parameters.

Parameter	Value	Rationale
Temperature	0.0 (greedy)	Deterministic outputs
Top-p	1.0 (disabled)	No sampling with greedy decoding
Max output tokens	512	Sufficient for full response; caps generation
Context window	4096 tokens	Standard for 7B–8B; sufficient for 5-turn conversations
Quantization (local)	Q4_K_M	Best quality-to-VRAM ratio for 8 GB GPU
Repetitions	1 per variant	Greedy decoding is deterministic

5 Planning

The project runs from June 2026 through October 2026. Artefact 1 is completed before Artefact 2 integration begins. Single-turn baseline experiments are completed before multi-turn experiments begin, so that unexpected baseline results can be addressed before the full experimental design is executed.

Table 7: Project timeline: shaded cells indicate active work periods.

Task	Jun	Jul	Aug	Sep	Oct
Deep literature review and gap formalisation	•				
Artefact 1: Mutation Engine, design and implementation		•	•		
Artefact 1: Trigger word detection pipeline		•	•		
Artefact 2: PyRIT integration, single-turn mode			•	•	
Artefact 2: Multi-turn orchestration (Strategy A & B)			•	•	
Experiments 1 & 2, single-turn baselines			•	•	
Experiments 3a & 3b, multi-turn strategies				•	•
Statistical analysis and results write-up				•	•
Final revisions and submission					•

Tasks on the critical path are Artefact 1 completion (including the trigger word detection pipeline, which must precede any experiment), the Experiment 1 baseline (which must precede Experiments 2 and 3), and the final submission in October. The estimated compute of 150–200 hours is scheduled as overnight runs across August–September, which is within the available window. The June slot is used for literature consolidation before implementation begins in July.

References

[Bou22] N. Boucher, I. Shumailov, R. Anderson, and N. Papernot. Bad Characters: Imperceptible NLP Attacks. In *Proc. 43rd IEEE Symposium on Security and Privacy (S&P)*, 2022. DOI: 10.1109/SP46214.2022.9833641.

[Bro20] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, and D. Amodei. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[Ebr18] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou. HotFlip: White-Box Adversarial Examples for Text Classification. In *Proc. 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.

[Gao25] Y. Gao et al. Imperceptible Jailbreaking Against Large Language Models. arXiv preprint, 2025.

[Goo15] I. Goodfellow, J. Shlens, and C. Szegedy. Explaining and Harnessing Adversarial Examples. In *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[Gra26] M. Graves. Reverse CAPTCHA: Evaluating LLM Susceptibility to Invisible Unicode Instruction Injection, 2026.

[Cha24] P. Chao, E. Debenedetti, A. Robey, M. Andriushchenko, F. Croce, V. Sehwag, E. Dobriban, N. Flammarion, G. J. Pappas, F. Tramer, H. Hassani, and E. Wong. JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models. arXiv:2404.01318, 2024.

[Sch26] L. Schwinn, M. Ladenburger, T. Beyer, M. Mofakhami, G. Gidel, and S. Günnemann. A Coin Flip for Safety: LLM Judges Fail to Reliably Measure Adversarial Robustness. arXiv:2603.06594, 2026.

[McN47] Q. McNemar. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[Sar25] E. Sarabamoun. Special-Character Adversarial Attacks on Open-Source Language Models, 2025.

[Sin24] S. Singh and A. S. Namin. A Survey on Chatbots and Large Language Models: Testing and Evaluation Techniques. *IEEE Access*, 2024.

[Sze14] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. Intriguing Properties of Neural Networks. In *Proc. International Conference on Learning Representations (ICLR)*, 2014.

[Che26] W. Chen, X. Meng, C. Zang, L. Wang, X. Gao, J. Wang, P. Zhan, Z. Li, and S. Guo. Not All Tokens Are Created Equal: Query-Efficient Jailbreak Fuzzing for LLMs. arXiv:2603.23269, 2026.

[Vas17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[Wun24] Wunderwuzzi. ASCII Smuggling and Hidden Prompt Instructions. *Embrace The Red*, 2024.

[Zou23] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv:2307.15043, 2023.

[Owi26] H. Owiredu-Ashley. ADVERSA: Measuring Multi-Turn Guardrail Degradation and Judge Reliability in Large Language Models. arXiv:2603.10068, 2026.

[Red25] A. Reddy, A. Zagula, and N. Saban. AutoAdv: Automated Adversarial Prompting for Multi-Turn Jailbreaking of Large Language Models. arXiv:2511.02376, 2025.

[Kum25] R. Kumar et al. With Great Backbones Comes Great Adversarial Transferability. arXiv:2501.12275, 2025.

[Zhe25] Y. Zheng et al. Enabling Heterogeneous Adversarial Transferability. arXiv:2503.20310, 2025.

[Yin26] H. Yin et al. Efficient Refusal Ablation in LLM through Optimal Transport. arXiv:2603.04355, 2026.

[Ji25] Q. Ji et al. The Geometry of Refusal in Large Language Models. arXiv:2502.17420, 2025.

[Kum25b] A. Kumarappan and A. Mujoo. Automating Deception: Scalable Multi-Turn LLM Jailbreaks. arXiv:2511.19517, 2025.

[Li26] P. Li, J. Zhang, T. Zhang, H. Qiu, K. Zhang, W. Zhang, N. Yu, and W. Zhou. State-Dependent Safety Failures in Multi-Turn Language Model Interaction. arXiv:2603.15684, 2026.

[Zha23] Y. Zhao et al. A Survey on Transferability of Adversarial Examples Across Deep Neural Networks. arXiv:2310.17626, 2023.